QAI

AI-Driven QA Desktop App

User Manual

Upload runs · Openworld agents · Pixel diffing · Parameterized tests · Skills · Reports

Version 2.0

Contents

1. Introduction ____ 2
   1.1 What QAI can do ____ 3
   1.2 Key terms ____ 3
2. Getting Started ____ 4
   2.1 Launching QAI ____ 5
   2.2 The main screen ____ 5
   2.3 First-time setup: connect an AI model ____ 5
3. Settings Reference ____ 6
   3.1 Global Settings ____ 7
   3.2 Advanced Settings ____ 7
   3.3 Backup & Reset ____ 8
4. Skills: Give the AI Your Own Knowledge ____ 8
   4.1 Loading a skill ____ 9
   4.2 Managing skills ____ 9
5. Upload Runs (Closed-World Testing) ____ 9
   5.1 How AI runs and script runs work together ____ 10
   5.2 Preparing the task file ____ 10
      Example task file ____ 11
   5.3 Uploading and editing the file ____ 11
   5.4 Running the tasks ____ 12
   5.5 What you get in the report ____ 12
6. Pixel Diffing (Visual Comparison) ____ 13
   6.1 Turning it on ____ 14
   6.2 Running a visual comparison ____ 14
   6.3 Reading the comparison ____ 14
7. Parameterized Tests (Data-Driven Replays) ____ 14
   7.1 Creating a parameterized case ____ 15
   7.2 Reviewing parameters and rows ____ 15
   7.3 Running and viewing results ____ 15
   7.4 Managing cases ____ 15
8. Openworld (Open-World Testing with AI Agents) ____ 16
   8.1 Screen layout ____ 17
   8.2 Walkthrough: from website to passing tests ____ 17
      Step 1: Plan ____ 17
      Step 2: Generate tests ____ 17
      Step 3: Run and heal ____ 18
   8.3 Talking to the agents ____ 18

8.4 Attaching files 18
8.5 Artifacts: viewing and editing files 18
8.6 Test cases & results panel 19
8.7 Usage cost 19
8.8 Openworld settings and chat backup 19
8.9 Running Openworld scenarios as Upload tasks 19
9. Working with the Report 20
9.1 Finding results 21
9.2 The test result page 21
9.3 Selecting, deleting and retrying results 21
9.4 Attachments reference 21
9.5 Failure types 22
9.6 Graphs and Timeline 22
10. Troubleshooting 22
Getting more detail 23
Appendix: Quick Reference 23
Common tasks at a glance 24

1. Introduction

QAI is a desktop application that tests websites with AI. You describe what to test in plain language; an AI agent drives a real browser to carry it out, records what it did as a reusable script, and writes the outcome to an interactive test report.

1.1 What QAI can do

Feature	What it does	Where in the app
Upload runs (closed-world)	Run a list of test tasks from a CSV or Excel file. Each task is performed by an AI agent the first time and can be replayed cheaply as a recorded script afterwards, in Chrome, Firefox and Safari.	**Upload** section
Openworld	Chat with three AI agents: the **Planner** explores your website and writes a test plan, the **Generator** turns it into Playwright tests, and the **Healer** runs the tests and fixes the ones that break.	**Openworld** section
Pixel diffing	Compares the screenshots of each step of a script replay with the previous replay and highlights visual changes.	Settings > Advanced Settings
Parameterized tests	Replays a recorded script with many rows of test data (boundary values, equivalence classes, pairwise combinations) generated by AI.	**Parameterize** button on a test result; **Parameterized** button on the Upload tab
Skills	Gives the AI your own instructions, codebase and reference documents to work from.	**Skills** button (Upload tab and Openworld message box)
AI failure analysis	After a run, each failed test is labelled with a failure type (Product bug, Test issue, Environment issue, ...) with a reason.	Test result page and **Categories**
Cost tracking	Shows the AI usage cost of each test, group and Openworld chat.	Results tree, test result header, Openworld chat header
Report	Results tree, history, retries, graphs and timeline, with multi-select delete and retry.	**Report**, **Graphs**, **Timeline** sections
Backup & restore	Back up and import your results, settings and Openworld chats; reset to defaults.	Settings > Backup & Reset; Openworld settings

1.2 Key terms

Term	Meaning
Task	One row of an uploaded task file: an instruction for the AI, such as "Log in and add user to the list".
AI run	The AI agent performs the task in a browser. Tagged **AI** in the report. It also records a script.
Script run	The recorded script of an earlier successful AI run is replayed without AI. Faster, cheaper, repeatable. Tagged **SCRIPT**.
Parameterized run	A script replayed with a row of test data. Tagged **PARAM**.
Closed-world testing	You state exactly which tasks to test (the Upload tab).
Open-world testing	The AI explores the application and decides what to test (the Openworld tab).
Skill	A file or folder you load so the AI can read it before working.
Artifact	A file produced or used in an Openworld chat: test plans, test scripts, attached documents.

2. Getting Started

2.1 Launching QAI

1. Open the **QAI** application.
2. Wait for the window to load. In the background QAI starts its report server and its QA backend. Until the backend is ready, run buttons show **Please wait…** and some panels show **Waiting for the QA backend to start...**. This normally takes a few seconds.

2.2 The main screen

The top of the window is the header:

- **Section picker** (top left). A drop-down used to switch between the five sections of the app: **Report**, **Graphs**, **Timeline**, **Upload** and **Openworld**.
- **Header controls** (top right): environment picker, layout toggle, theme (light/dark) button, and the **gear icon**, which opens Settings.

Section	Purpose
Report	The test report: results tree on the left, the selected test result on the right.
Graphs	Charts of status, durations, stability and trends.
Timeline	Every test drawn on a timeline, useful for parallel runs.
Upload	Upload a task file and run it (closed-world testing).
Openworld	Chat with the Planner, Generator and Healer agents (open-world testing).

> **Note:** Switching to **Report**, **Graphs** or **Timeline** refreshes the report so that new results appear (you will see a short loading overlay). Switching to **Upload** or **Openworld** does not refresh, so a loaded file or an ongoing chat is kept. While a run is in progress the section picker and the gear icon are disabled.

2.3 First-time setup: connect an AI model

QAI needs an AI (LLM) provider and API key before it can run anything.

1. Click the **gear icon** at the top right. The **Global Settings** dialog opens.
2. In **LLM Provider**, choose your provider: Siraya, Gemini, OpenAI, Anthropic, DeepSeek, Kimi, AWS Bedrock or OpenAI Compatible.
3. Enter your credentials:
   - Most providers: paste the key into **API Key**.
   - **OpenAI Compatible**: also enter the **Base URL** of the service.
   - **AWS Bedrock**: enter **AWS Region**, **AWS Access Key ID** and **AWS Secret Access Key**.
4. In **Model Name**, type a model or click the ▼ arrow to pick from the list QAI loads from your provider (not available for AWS Bedrock; type the model id).
5. Optionally open **Advanced Settings** (see Chapter 3) to choose headless mode, parallel mode, pixel diffing and more.

6. Click **Save**. QAI restarts its backend to apply the settings; run buttons show **Please wait...** for a moment.

> **Tip:** Each provider remembers its own key and model, so you can switch between providers without re-typing them.

3. Settings Reference

Open Settings with the **gear icon** in the header. The dialog has three pages: **Global Settings**, **Advanced Settings** and **Backup & Reset**. Every **Save** restarts the QA backend, because it only reads settings when it starts.

3.1 Global Settings

Setting	Description
LLM Provider	The AI provider used by Upload runs, Openworld agents, parameterized data generation and failure analysis.
API Key / AWS fields / Base URL	Credentials for the chosen provider (see 2.3).
Model Name	The model to use. The ▼ list is loaded live from the provider once the key is entered.

3.2 Advanced Settings

Click **Advanced Settings** on the Global Settings page. Click **Back** to return, **Save** to apply.

Setting	Description
Max Actions Per Step	How many browser actions the AI may take in one step (1–50). Default: 30.
Max Steps	The maximum number of steps an AI run may take (1–500). Also limits how far the Openworld Planner explores.
Developer Mode	Opens a separate **QAI Developer Mode** log window showing what the backend is doing.
Headless Mode	On: browsers run invisibly. Off (default): you can watch the browser while the AI works and while scripts replay.
Parallel Mode / Number of Workers	Run several tests at the same time. Choose **Auto** (one per CPU core) or a fixed number. Off: tests run one by one.
Pixel Diffing	Turns on step-by-step screenshot comparison for Upload tasks (see Chapter 6).
Load Browser Extension / Browser Extension Folder Path	Load unpacked browser extensions into the test browser. Click **Browse** and pick a folder that contains `chrome`, `firefox` and `safari` sub-folders, each holding at least one extension folder with a valid `manifest.json`.
Browser Type (SCRIPT)	Default browser (Chrome, Firefox or Safari) for script replays and parameterized replays when a task does not name one.
Script Validation (Regex Method / LLM Method)	After an AI run, the recorded script is replayed once to make sure it works. If it fails, QAI repairs it with pattern rules (Regex) and/or the AI (LLM). The original recording is kept as an extra attachment.

Setting	Description
Retry after failed / Number of retries	Automatically re-run a failed AI task up to 1–10 times.
Failure Classification	After each run, label every failed test with a failure type and reason (on by default). See 9.5.
Browser Use + Jev	Uses the Jev Ultrafast agent for a faster first pass of AI runs and for failure classification. Requires a **TypeSafe API Key**; optionally set **TypeSafe Model** and **Text Model Reasoning** (None/Low/Medium/High).

3.3 Backup & Reset

Click **Backup & Reset** on the Global Settings page.

Button	What it does
Backup data	Saves all test results to a `.qaidata` file.
Import data	Loads a `.qaidata` file. **Replaces** all current results.
Backup settings	Saves your settings (including API keys) to a `.qaiconfig` file.
Import settings	Loads a `.qaiconfig` file, replacing current settings.
Delete data	Deletes all test results (asks for confirmation).
Reset settings	Restores default settings (asks for confirmation).
Factory reset	Deletes all results and all settings (asks for confirmation).

> **Important:** Import data and the reset buttons cannot be undone. Back up first. These backups do not include Skills, parameterized cases or Openworld chats; Openworld chats have their own backup (see 8.8). A `.qaiconfig` file contains your API keys, so store it safely.

4. Skills: Give the AI Your Own Knowledge

A skill is a file or folder that the AI reads before it works. Use skills to give the AI your test conventions, the source code of the application under test, or reference documents such as requirements.

Skill type	What you load	How the AI uses it
Instructions	A single `.md`, `.txt`, `.pdf` or `.docx` file	Rules to follow. If an instruction conflicts with the task, the task wins.
Codebase	A whole folder	Facts about the application and its tests.
Reference	Any other single file (e.g. `.json`, `.csv`, `.xlsx`)	Facts to consult.

4.1 Loading a skill

1. Click **Skills** on the Upload tab, or the **Skills** pill under the Openworld message box. The **Skills** dialog opens.
2. Under **Load a skill**, click **Choose files or a folder** and select one or more files or a folder. You can also drag and drop files and folders onto this area (on Windows/Linux, folders must be dragged in).
3. Wait for the confirmation, e.g. **Loaded X as instructions.**
4. In **Loaded skills**, tick **Upload** and/or **Openworld** to decide where each skill is used. New skills are switched on for both.
5. Click **Done**.

4.2 Managing skills

- The Skills button shows how many skills are on, e.g. **Skills (2)**.
- To remove a skill, click its trash icon. It is removed immediately.
- A skill is a copy taken when you load it. If the original files change, load the skill again.
- Changes apply from the next test or chat message.
- When loading a folder, QAI skips dependency and build folders (e.g. `node_modules`, `.git`, `dist`), binaries, logs and files that may contain secrets (e.g. `.env`, `*.pem`, `*.key`). Limits: 5000 files / 100 MB per folder; 2 MB per text file, 20 MB per PDF/DOCX/XLSX.

> **Tip:** To use different skills for different tasks in one Upload file, add a **Skill** column to the file (see 5.2).

5. Upload Runs (Closed-World Testing)

In the Upload tab you give QAI a list of tasks in a CSV or Excel file. QAI runs each task and writes the results to the report.

5.1 How AI runs and script runs work together

The first time a task runs, the AI agent performs it in Chrome while QAI records every browser action as a Playwright script. When that AI run passes, the script is attached to the result. On later runs, if **Copilot Mode** is FALSE, QAI replays the script instead of using AI: faster, cheaper, consistent, and available in Chrome, Firefox and Safari.

Copilot Mode	Script recorded for this task?	What happens	Report tag
TRUE	Any	The AI performs the task (Chrome) and records a new script.	AI
FALSE	No	The AI performs the task and records a script.	AI
FALSE	Yes	The latest passing recorded script is replayed in each listed browser, without AI.	SCRIPT

> **Note:** A script is matched to its task by the task ID, the file name (extension ignored) and the exact values of task_content, category, title, description, base_url and credentials. If you rename the file or change any of these, no script is found and the task runs with AI again. Scripts from failed AI runs are never replayed.

> **Tip:** Recommended workflow: run a new task once with Copilot Mode TRUE, check it passes, then set Copilot Mode to FALSE for all later regression runs.

5.2 Preparing the task file

Create a `.csv` or `.xlsx` file (only the first worksheet of an Excel file is used). The first row holds column names. Column names are not case-sensitive, and spaces, hyphens and underscores are treated the same, so `Task Content`, `task-content` and `task_content` are equivalent. Only `task_content` is required; `task_id` is strongly recommended.

Column	Required?	Description
task_id	Recommended	A short ID, e.g. `1` or `LOGIN-01`. Shown in the result title "Task {id}: ..." and used to find the recorded script.
task_content	**Yes**	The instruction for the AI, in plain language. Example: `Log in as demo/demo123 and add the first product to the cart`.
base_url	No	The page to start from. Default: Google.

Column	Required?	Description
expected_result	No	What should happen. The AI checks it and an AI judge reviews the run against it. Example: `The cart shows 1 item`.
expected_keywords	No	Text that must appear in the AI's final answer. `|` means OR, `&` means AND, e.g. `Welcome & Logout | Dashboard`. Case-sensitive.
credentials	No	Login details or other data the AI may use.
title	No	Result title. Default: the first 40 characters of task_content.
description	No	Result description.
category	No	Groups results in the report tree under the file name. Default: `TestQA`.
Copilot Mode	No	TRUE = always use AI. FALSE (or empty) = replay the recorded script when one exists.
Browser Type	No	`chrome`, `firefox`, `safari`, or several separated by commas (in quotes in a CSV: `"chrome,firefox"`). Applies to script replays; AI runs always use Chrome. Empty = the **Browser Type (SCRIPT)** setting.
Skill	No	Skills for this task only, comma-separated (by name, e.g. `login-rules.md`, or by the path they were loaded from). If this column exists, an empty cell means **no skills**. Without the column, every task uses the skills switched on for Upload.
extension_path	No	Folder of a browser extension for this task. If the column exists, an empty cell means no extension.
llm_provider, model_name	No	Override the AI provider/model from Settings for this task (e.g. `openai`, `gemini`, `anthropic`).

Example task file

```
task_id,task_content,base_url,expected_result,Copilot Mode,Browser Type
1,Log in with user demo and password demo123,https://shop.example.com,The dashboard
is shown,TRUE,chrome
2,Search for "laptop" and open the first result,https://shop.example.com,A product
page is shown,FALSE,"chrome,firefox,safari"
3,Add the first product to the cart,https://shop.example.com,The cart shows 1
item,FALSE,
```

> **Tip:** Write each task as one self-contained user journey and state a clear, checkable expected result. Put login details in the credentials column or the task itself.

5.3 Uploading and editing the file

1. Select **Upload** in the section picker.

2. (Optional) Click **Skills** to load or switch on skills (Chapter 4).
3. Click **Upload File** and choose your `.csv` or `.xlsx` file.
4. The file appears as an editable table, with the file name and **X of Y tasks selected** at the top.
5. Review and edit the tasks:
    - Click any text cell to edit it.
    - **Copilot Mode** cells are a checkbox (TRUE/FALSE).
    - **Browser Type** cells have **chrome**, **firefox** and **safari** checkboxes.
    - **Skill** cells have one checkbox per loaded skill. A skill named in the file but not loaded shows as "(not loaded)" and the task runs without it.
    - Untick a row's checkbox to leave it out of this run; the header checkbox selects or clears all rows.
6. (Optional) Click **Download** to save your edited table as a CSV for next time.

> **Note:** When a file loads, QAI tidies it: empty Copilot Mode cells become FALSE, unknown browser names are dropped, and completely blank rows are removed. Click **Clear** to remove the file and start again.

5.4 Running the tasks

1. Click **Run**. Only the ticked rows are run.
2. Watch the progress panel above the button. It shows:
    - The current stage, e.g. **Running test 3 of 10**.
    - Elapsed time and an estimate of the time left.
    - The task that is running now, e.g. "Task 3: Log in (firefox)".
    - Counts of passed, failed, error and skipped tests, and a coloured progress bar.
3. To pause, click **Pause**. Tests already running finish first, then the run waits ("Paused before test N of M"). Click **Resume** to continue.
4. To stop, click **Cancel**. Results already finished are kept.
5. When the run finishes, the panel shows **Completed N tests** with the totals (green when all passed, red otherwise).
6. Select **Report** in the section picker to see the results (Chapter 9).

> **Note:** If **Headless Mode** is off, you can watch the browser while the AI works. While a run is in progress you cannot switch sections or open Settings. After the tests, QAI also runs the AI failure analysis; the panel shows **Finishing...** during this time.

5.5 What you get in the report

- Results are grouped by uploaded file name, then category, then "Task {id}: {title}".
- Each AI step has **Agent Thoughts**, **URL**, **Step Duration**, **Screenshot** and **Usage** attachments, so you can follow exactly what the AI did and why.
- The run ends with a **Final Result** attachment and, for a passed AI run, **Script (...)**, the recorded Playwright script.
- Script replays show each Playwright action as a step with a screenshot.

- Tags show the run type (**AI** or **SCRIPT**) and the browser (**CHROME**, **FIREFOX**, **SAFARI**).
- The cost of each test is shown next to its duration.

6. Pixel Diffing (Visual Comparison)

Pixel diffing compares the screenshots of each step of a script replay with the screenshots of the previous script replay of the same task, and highlights what changed visually. Use it to catch unexpected layout or styling changes between releases.

6.1 Turning it on

1. Click the **gear icon**, then **Advanced Settings**.
2. Tick **Pixel Diffing**.
3. Click **Save**.

6.2 Running a visual comparison

With Pixel Diffing on, every task with a **base_url** is a pixel-diff task. These tasks run one at a time after the other tasks, and once a script has been recorded they always replay the script (even if Copilot Mode is TRUE). A comparison needs three runs of the same task:

Run	What happens
1st run	AI run. The AI performs the task and records a script.
2nd run	Script replay. This becomes the baseline.
3rd and later runs	Script replay, compared step by step with the most recent earlier script replay.

1. Upload a task file whose tasks have a **base_url** and click **Run** (1st run).
2. Click **Run** again (2nd run, baseline).
3. Click **Run** again whenever you want to check for visual changes (3rd run onwards).
4. Go to **Report**, open the latest SCRIPT result of the task, and expand its steps.
5. Open the **Pixel Diff Screenshot** attachment on a step.

6.3 Reading the comparison

The Pixel Diff Screenshot attachment opens in an image viewer with these tabs:

Tab	Shows
Diff	The difference image; changed pixels are red.
Expected	The **current** run's screenshot, with red boxes around the changed areas.
Actual	The **previous** script run's screenshot.
Side by side	Both screenshots next to each other.
Overlay	One screenshot over the other.

> **Note:** In this viewer, "Expected" is the current run and "Actual" is the previous run. Steps are paired by position (1st step with 1st step, and so on), so the comparison is most useful when the script is unchanged. A visual difference never fails a test; it is information for you to review.

7. Parameterized Tests (Data-Driven Replays)

A parameterized test takes the script an Upload AI run recorded and replays it many times with different data, for example different amounts, emails or dates. The AI suggests which typed or selected values should become parameters and proposes test values; you can edit everything before running.

7.1 Creating a parameterized case

1. Run an Upload task with AI so that it passes and records a script.
2. Go to **Report** and open that test result.
3. Click **Parameterize** in the result header (next to the duration and cost). The **Parameterized tests** dialog opens.
4. Under **Generate data rows**, choose the techniques to use (all are ticked by default):
    - **Boundary values**: values at each edge of what a field accepts, and just beyond it.
    - **Equivalence classes**: one value for each kind of input the application treats alike, valid and invalid.
    - **Pairwise**: every pair of accepted values of two parameters, in as few rows as possible.
5. (Optional) Describe the data in the hint box, e.g. "amount is a whole number from 1 to 1000".
6. Click **Generate**. The AI picks parameters and fills the table with data rows.

7.2 Reviewing parameters and rows

- Each parameter is a column. Click its name to rename it; click **×** to remove it (the recorded value is then used).
- **+ Add parameter** turns another typed or selected value from the script into a parameter.
- **Add row** adds a manual row, pre-filled with the recorded values.
- For each row set **Expected** to **Accepted** (the flow should complete) or **Rejected** (the application should refuse the data). Optionally fill **Page shows** with text that must be visible afterwards, and a **Note**.
- The first row, **Recorded**, holds the original values and cannot be changed.
- Untick a row to leave it out of the run. A coloured dot shows each row's last result.
- Changes save automatically (**Saved** appears at the bottom). Generating again replaces earlier rows of the same technique but keeps the rows you added by hand.

7.3 Running and viewing results

1. Click **Run N rows**. Progress is shown as in the Upload tab; **Cancel** stops the run.
2. When finished, click **Show results** to refresh the report.
3. In the report, each row is a separate result tagged **PARAM**, titled with the values that differ, e.g. `Task 1: Pay [amount='0']`.

> **Rule:** A row passes when the application does what is expected: with accepted data the recorded steps complete; with rejected data they stop once the data is entered (shown in a **Where the flow stopped** attachment).

7.4 Managing cases

Click **Parameterized** on the Upload tab to see all cases with their parameter, row and pass/fail counts. Click a case to open it, or its trash icon to delete it (its results stay in the report). Replays use the **Browser Type (SCRIPT)**, **Parallel Mode** and **Headless Mode** settings.

8. Openworld (Open-World Testing with AI Agents)

In Openworld you chat with three AI agents that together plan, write and maintain automated tests for your website:

Agent	What it does	Output
Planner	Opens a real browser, explores your website (filling in forms and trying flows), and writes a test plan.	A Markdown test plan (`.plan.md`) of suites and numbered scenarios, e.g. 1.1, 1.2, 2.1.
Generator	Performs each planned scenario live in the browser, checks every expected result, and writes a test for it.	One Playwright (Python) test file per scenario.
Healer	Runs the tests. For each failure it diagnoses the cause and either fixes the test or marks it **fixme** when the application itself is broken.	Test results in the report; fixed versions of test files.

8.1 Screen layout

- **Sidebar** (left): **New chat**, **Artifacts**, your chat list grouped by date, and **Settings** at the bottom.
- **Chat area** (middle): the conversation, with the message box at the bottom. Under the message box are the **agent picker**, the **Skills** pill, the **paperclip** (attach files) and the **send** arrow.
- **Test cases & results panel** (right): this chat's plan scenarios and tests with their latest status. Toggle it with the panel icon in the chat header.
- **Usage pill** (chat header, coin icon): the chat's AI cost so far.

8.2 Walkthrough: from website to passing tests

Step 1: Plan

1. Select **Openworld** in the section picker and click **New chat**.
2. Click the agent picker under the message box and choose **Planner** (the default). You can also click the **Plan a website** suggestion card and replace `https://...` with your URL.
3. Type a message describing the app, its URL, any login details and the flows to cover. Example: "Plan tests for https://shop.example.com. Log in with demo / demo123. Cover search, cart and checkout."
4. (Optional) Attach requirement documents with the **paperclip** or by dragging them onto the message box.
5. Press **Enter** to send (Shift+Enter for a new line).
6. Watch the progress: the status line (e.g. **Exploring the application**) and the expandable browser log (**Working in the browser...**) show each page visited and the AI's reasoning.
7. When done, a plan card appears: "Saved the test plan X.plan.md (v1): N scenarios in M suites." Click **Show scenarios** to read it.

> **Tip:** To change the plan without re-exploring, ask the Planner, e.g. "Make 2.1 stricter" or "Remove 3.2". You can also edit the plan yourself in Artifacts.

Step 2: Generate tests

1. Click **Generate tests →** under the plan. This switches to the **Generator** and fills in "Generate tests for every scenario in the plan."
2. Edit the message to target specific scenarios if you wish, e.g. "Generate 1.1 and 2.3" or "Generate suite 2".
3. Press **Enter**. For each scenario the Generator performs the steps in the browser and writes a test. Each test row shows a status badge (Generating, Generated, ...).
4. Click **Open** on a test row to view its code in Artifacts.

> **Note:** The Generator does not run the tests. Their status is **Generated** ("Not run yet") until the Healer runs them.

Step 3: Run and heal

1. Click **Run and heal the tests →**. This switches to the **Healer** and fills in "Run all tests and fix the failing ones." Press **Enter**.
2. The Healer runs all tests. For each failure it tries up to 3 fixes, saving each as a new version, then runs the changed tests once more for the report.
3. Read the summary, e.g. "Ran 5 tests: 3 passed, 1 fixed by the healer, 1 marked fixme." A **fixme** test indicates a likely application bug.
4. Click **Open report** to see the results in the report.

> **Tip:** To only run the tests without changing them, say "Just run the tests, don't fix anything." If failures remain after a run-only turn, click **Heal the failing tests →**.

8.3 Talking to the agents

- Each message goes to the agent selected in the agent picker. Each chat remembers the last agent used.
- You can also just ask an agent a question; it answers without opening a browser.
- Refer to scenarios by ID ("1.2") or suite ("suite 2") to limit the work.
- Click the ■ **Stop** button (in place of the send arrow) to stop a running turn.
- Only one turn runs at a time in a chat; while it runs, New chat, switching chats and the message box are locked. Openworld messages cannot be sent while an Upload run is in progress.

8.4 Attaching files

Click the **paperclip**, drag files onto the message box, or paste them.
Supported: `.md` `.txt` `.csv` `.xlsx` `.json` `.py` `.pdf` `.docx` `.png` `.jpg` `.jpeg` `.webp` `.gif`; up to 10 files per message and 20 MB each. URLs cannot be attached; type them in the message.

- **Planner**: requirements, specs or screenshots to plan from.
- **Generator**: an existing plan (Markdown, or a CSV/Excel file of test cases) to generate tests from.
- **Healer**: existing Python tests named `test_*.py` or `*_test.py` to run and heal.

8.5 Artifacts: viewing and editing files

Click **Artifacts** in the sidebar to see every file from every chat, grouped by chat and by type (Test plans, Tests, Test suites, Files). Click it again to return to the chat.

- Click a file to open it. Text files (`.md`, `.py`, `.txt`, `.json`) open in a text editor; `.csv`/`.xlsx` open in a spreadsheet editor; images are previewed.
- Edit the file, then click **Save as vN+1**. Every save creates a new version; nothing is overwritten. **Discard** throws away your changes. Unsaved changes are lost if the app reloads.
- The **Versions** pane lists all versions and who made them (Generated by an agent, Edited, Healed by the healer, ...). **Restore** brings back an older version as a new version.
- Use the ⋮ menu to **Rename**, **Duplicate**, **Download** or **Delete** a file. **Upload file** adds a file of your own to a chat.
- Agents always use the latest version of a file.

8.6 Test cases & results panel

- Shows the plan's suites and scenarios with their status, plus tests that are not in the plan.
- Filter with **All**, **Changed** or **Failing**; switch between the **Test cases** and **Files** tabs.
- Click a scenario to see its steps, the test file with the failing line highlighted, the Healer's diagnosis, and its history in the chat. **Open in Artifacts** opens the file for editing.
- The top line shows the last run, with **Output**, **Open report** and **Show in chat** links.

8.7 Usage cost

The coin pill in the chat header shows the chat's total AI cost (e.g. `$0.0123`). Click it for a breakdown by input, cached input and output tokens, by model and by agent. Costs are estimates; your provider's bill may differ.

8.8 Openworld settings and chat backup

Click **Settings** at the bottom of the sidebar.

Option	Description
Report category	Name under which this chat's test results are grouped in the report (default `openworld`).
Backup chat history	Saves all chats and their artifacts to a `.qaichats` file.
Import chat history	Adds the chats from a `.qaichats` file. Existing chats are kept.
Clear all chat history	Deletes every chat and all artifacts (asks for confirmation).

To rename or delete a single chat, click the ⋮ next to it in the chat list.

8.9 Running Openworld scenarios as Upload tasks

Under a Planner reply (or on a spreadsheet in Artifacts), click **Open in Upload tab**. The plan's scenarios are loaded into the Upload tab as AI tasks, ready to review and **Run** as described in Chapter 5.

9. Working with the Report

9.1 Finding results

- Select **Report** in the section picker. The results tree is on the left; click a test to open it on the right.
- Type in **Search tests** to find tests by name.
- Use the status tabs (**Total**, Passed, Failed, Broken, Skipped, Unknown) and the quick filters (**Retries**, **Flaky**, **Transition**, **Tags**, **Categories**) to narrow the list.
- Use **Sort by** to order by Order, Alphabet, Duration or Status.
- Each row shows the failure type tag, cost, duration, run type (AI / SCRIPT / PARAM) and browser. Group headers show the total cost of the group.

9.2 The test result page

- The header shows status, failure type, run type, browser, duration, **cost** and the **Parameterize** button (when the test recorded a script).
- Tabs: **Overview** (steps and attachments), **History** (earlier runs of the same test), **Retries**, **Attachments**, **Environments**.
- For failed tests, the **AI failure analysis** box shows the failure type, confidence (high/medium/low) and the reason.

9.3 Selecting, deleting and retrying results

1. Click **Select** above the results tree. Checkboxes appear on every test and group.
2. Tick the tests to act on (a group checkbox selects the whole group).
3. Click **Retry** to run them again, or **Delete** to remove them, then confirm with **Yes**.

- **Retry** re-runs AI results with AI, SCRIPT results as script replays (in the browser originally used) and PARAM results through their parameterized case. A summary dialog shows the outcome; click **OK** to refresh the report. Openworld results cannot be retried here; use the Healer instead.
- **Delete** removes the selected results together with all their earlier runs and retries.

9.4 Attachments reference

Attachment	Contents
Agent Thoughts	The AI's reasoning for a step.
URL / Step Duration / Screenshot	Page address, time taken and screenshot for a step.
Usage	Tokens used and cost.
Final Result	The AI's final answer for the task.
Script (...)	The recorded Playwright script; used for script replays, retries and parameterized tests.
Script (...) (Original)	The recording before Script Validation repaired it.
Modified Script	The script as replayed in a SCRIPT run.

Attachment	Contents
Skills	Which skills the AI used (and any that were named but not loaded).
Failure Classification	Failure type, confidence and reason.
Pixel Diff Screenshot	Visual comparison with the previous script run (Chapter 6).
Where the flow stopped	For a rejected parameterized row, where the application stopped the flow.
Generated test (file.py)	The test code of an Openworld result.
Uploaded CSV file / Uploaded Excel file	The task file used for the run.

9.5 Failure types

When **Failure Classification** is on, every failed or broken test is labelled with one of these types. Each type also appears as a group in the **Categories** tab.

Failure type	Meaning	Typical action
Product bug	The application under test misbehaved.	Report the bug to developers.
Test issue	The test or a locator is wrong or outdated.	Update the task, plan or script; or use the Healer.
Test data issue	Missing or expired data or preconditions.	Fix the test data or account.
Environment issue	Site down, network problem, browser didn't start, missing API key, rate limit, AI outage.	Check connectivity and settings, then retry.
AI agent issue	The AI gave up, looped, ran out of steps or misread the task.	Clarify the task wording, raise Max Steps, retry.
Needs review	The evidence is inconclusive.	Inspect screenshots and steps manually.

9.6 Graphs and Timeline

- **Graphs** shows charts such as Current Status, Status dynamics, Status transitions, Durations histogram, Stability distribution, Test results by severities and Trend Chart.
- **Timeline** shows when each test ran and on which worker; drag across it to select a time range.

10. Troubleshooting

Problem / message	What to do
Buttons show **Please wait...** / "Waiting for the QA backend to start..."	The backend is starting (also after every Settings save). Wait a few seconds. If it persists, restart QAI.
"Agent failed to provide a final result. Did you provide an API key?"	Check the LLM Provider, API Key and Model Name in Settings.
Model list shows "Request failed (401)"	The API key is invalid or lacks access. Re-enter it.
"Please select a .csv or .xlsx file."	Only CSV and Excel files can be uploaded.
Run is disabled	No rows are ticked, the file has no tasks, or the backend is not ready.
New results don't appear in the report	Switch to **Report** in the section picker; this refreshes the report.
A task always runs with AI although Copilot Mode is FALSE	No passing script matches the task. Check the earlier AI run passed and that task_id, the file name and the task columns are unchanged.
"Did not meet expected conditions keywords"	The final answer lacked the expected_keywords text. Keywords are case-sensitive.
"Invalid Browser extension folder"	The folder needs `chrome`, `firefox` and `safari` sub-folders with valid extensions.
"Browser Use + Jev needs a TypeSafe API Key."	Enter the key or untick Browser Use + Jev.
Openworld: "There is no test plan in this chat yet."	Ask the Planner to write one, or attach a plan.
Openworld: "There are no tests in this chat yet."	Ask the Generator to write them, or attach `test_*.py` files.
Openworld: "No scenarios could be read from the plan"	Check the plan headings follow the format the Planner writes (suites as `### 1. Name`, scenarios as `#### 1.1. Name`).
Retry: "No retryable AI or SCRIPT task with the required result metadata was found"	The selected results cannot be retried from the tree (e.g. Openworld results). Use the Healer.
Linux: backend fails with `EACCES`	Run `export APPIMAGE_EXTRACT_AND_RUN=1` before launching, or make sure `/tmp` is not mounted `noexec`.

Getting more detail

Turn on **Developer Mode** in Advanced Settings to open a log window that shows what the backend is doing during runs.

Appendix: Quick Reference

Common tasks at a glance

I want to...	Do this
Set up the AI model	Gear icon > choose LLM Provider, API Key, Model Name > Save
Run a list of tests	Upload > Upload File > tick rows > Run > Report
Replay tests cheaply	Set Copilot Mode to FALSE for tasks that already passed with AI
Test in several browsers	Browser Type: chrome, firefox, safari (script replays)
Check for visual changes	Settings > Advanced > Pixel Diffing > Save; run tasks with base_url 3+ times; open Pixel Diff Screenshot
Test with many data values	Open a passed AI result > Parameterize > Generate > Run N rows
Let AI explore and write tests	Openworld > New chat > Planner > Generate tests → > Run and heal the tests →
Give the AI my docs or code	Skills > Choose files or a folder > tick Upload / Openworld
Re-run some failed tests	Report > Select > tick tests > Retry
Back up everything	Settings > Backup & Reset > Backup data + Backup settings; Openworld Settings > Backup chat history